

Doubly Linked List - Complete C Program

Source Code

```
#include <stdio.h>
#include <malloc.h>

struct Node
{
    struct Node *prev;
    int data;
    struct Node *next;
};

struct Node *head, *temp;

/* Function to insert node at beginning */
void insertNodeAtBegin(int ele)
{
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->next = NULL;
    newNode->prev = NULL;
    newNode->data = ele;

    if (head == NULL)
        head = newNode;
    else
    {
        newNode->next = head;
        head->prev = newNode;
        head = newNode;
    }
}

/* Function to insert node at last */
void insertNodeAtLast(int ele)
{
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->next = NULL;
    newNode->prev = NULL;
```

```

newNode->data = ele;

if (head == NULL)
    head = newNode;
else
{
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    newNode->prev = temp;
    temp->next = newNode;
}
}

/* Function to insert before a specific node */
int insertNodeAtSpec(int eletoInsert, int atInsert)
{
    struct Node *newNode = malloc(sizeof(struct Node));
    newNode->next = NULL;
    newNode->prev = NULL;
    newNode->data = eletoInsert;

    if (head == NULL)
        head = newNode;
    else
    {
        if (head->data == atInsert)
        {
            head->prev = newNode;
            newNode->next = head;
            head = newNode;
            return 1;
        }
        temp = head;
        while (temp != NULL && temp->data != atInsert)
            temp = temp->next;

        if (temp == NULL)
            return 0;
        else
        {

```

```

        (temp->prev)->next = newNode;
        newNode->prev = temp->prev;
        newNode->next = temp;
        temp->prev = newNode;
        return 1;
    }
}

```

/* Delete first node */

```

int deleteFirstNode()
{
    if (head == NULL)
        return 0;
    temp = head;
    head = head->next;
    if (head != NULL)
        head->prev = NULL;
    free(temp);
    return 1;
}

```

/* Delete last node */

```

int deleteLastNode()
{
    if (head == NULL)
        return 0;
    temp = head;
    while (temp->next != NULL)
        temp = temp->next;
    if (temp->prev != NULL)
        temp->prev->next = NULL;
    else
        head = NULL;
    free(temp);
    return 1;
}

```

/* Delete specific node */

```

int deleteSpecNode(int ele)
{

```

```

    if (head == NULL)
        return 0;
    temp = head;
    while (temp != NULL && temp->data != ele)
        temp = temp->next;
    if (temp == NULL)
        return 0;
    if (temp->prev != NULL)
        temp->prev->next = temp->next;
    else
        head = temp->next;
    if (temp->next != NULL)
        temp->next->prev = temp->prev;
    free(temp);
    return 1;
}

```

```

/* Count total nodes */
int countNumber()
{
    int count = 0;
    temp = head;
    while (temp != NULL)
    {
        count++;
        temp = temp->next;
    }
    return count;
}

```

```

/* Traverse list forward */
int traverse()
{
    if (head == NULL)
        return 0;
    temp = head;
    while (temp != NULL)
    {
        printf("\t%d", temp->data);
        temp = temp->next;
    }
}

```

```

        return 1;
    }

/* Search element */
int search(int ele)
{
    temp = head;
    while (temp != NULL)
    {
        if (temp->data == ele)
            return 1;
        temp = temp->next;
    }
    return 0;
}

/* Traverse reverse (recursively) */
int traverseReverse(struct Node *temp)
{
    if (temp == NULL)
        return 0;
    traverseReverse(temp->next);
    printf("\t%d", temp->data);
}

/* Main Menu */
int main()
{
    int choicel, choice2, ele, atInsert, count;
    do
    {
        printf("\n1.Insert an Element");
        printf("\n2.Delete an Element");
        printf("\n3.Traverse List");
        printf("\n4.Search Element");
        printf("\n5.Count Elements");
        printf("\n6.Reverse Traverse");
        printf("\n7.Exit");
        printf("\nEnter Your Choice: ");
        scanf("%d", &choicel);
    }

```

```

switch (choice1)
{
case 1:
    printf("\n1.Insert at beginning");
    printf("\n2.Insert at last");
    printf("\n3.Insert before specific");
    printf("\nEnter your choice: ");
    scanf("%d", &choice2);
    printf("\nEnter the element: ");
    scanf("%d", &ele);
    if (choice2 == 1) insertNodeAtBegin(ele);
    else if (choice2 == 2) insertNodeAtLast(ele);
    else
    {
        printf("\nInsert before element: ");
        scanf("%d", &atInsert);
        insertNodeAtSpec(ele, atInsert);
    }
    break;

case 2:
    printf("\n1.Delete first");
    printf("\n2.Delete last");
    printf("\n3.Delete specific");
    printf("\nEnter your choice: ");
    scanf("%d", &choice2);
    if (choice2 == 1) deleteFirstNode();
    else if (choice2 == 2) deleteLastNode();
    else
    {
        printf("\nEnter element to delete: ");
        scanf("%d", &ele);
        deleteSpecNode(ele);
    }
    break;

case 3:
    printf("\n\nList Elements:\n");
    if (!traverse()) printf("List is empty.");
    break;

```

```
case 4:
    printf("\nEnter element to search: ");
    scanf("%d", &ele);
    if (search(ele)) printf("\nElement Found.");
    else printf("\nElement Not Found.");
    break;

case 5:
    count = countNumber();
    printf("\nTotal Nodes: %d", count);
    break;

case 6:
    printf("\nReverse List:\n");
    traverseReverse(head);
    break;
}
} while (choicel != 7);
}
```

Output :

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 1

```
1.Insert at begin :  
2.inset at last :  
3.insert at spec :
```

Enter Yout Choice : 1

Enter the Element : 10

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 1

```
1.Insert at begin :  
2.inset at last :  
3.insert at spec :
```

Enter Yout Choice : 2

Enter the Element : 20

```
1.Insert an Element :  
2.Delete An Element :  
3.Traverse List :  
4.Search Element :  
5.Count of Element :  
6.Reverse Traverse :  
7.Exit :
```

Enter Your Choice : 1

```
1.Insert at begin :  
2.inset at last :  
3.insert at spec :
```

Enter Yout Choice : 3

Enter the Element to insert : 30

lement at insert : 20

```
1.Insert an Element :
```



```
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 3

====Content of the List====

10 30 20

```
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 4

Enter the Element to Search : 10

Element Found

```
1.Insert an Element :
2.Delete An Element :
```

```
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 5

Number of the Nodes 3:

```
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Element :
5.Count of Element :
6.Reverse Traverse :
7.Exit :
```

Enter Your Choice : 6

===Reverse Content of the List===

20 30 10

```
1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Element :
```

5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 3

====Content of the List====

10 30 20

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 2

1.delete at begin :
2.delete at last :
3.delete at spec :

Enter Yout Choice : 1

1.Insert an Element :
2.Delete An Element :
3.Traverse List :

4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 2

1.delete at begin :
2.delete at last :
3.delete at spec :

Enter Yout Choice : 2

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 3

====Content of the List====

30 20

1.Insert an Element :
2.Delete An Element :

3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 2

1.delete at begin :
2.delete at last :
3.delete at spec :

Enter Yout Choice : 3

Enter the Element to Delete : 30

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 3

====Content of the List====

====Content of the List====

20

1.Insert an Element :
2.Delete An Element :
3.Traverse List :
4.Search Ellement :
5.Count of Element :
6.Reverse Traverse :
7.Exit :

Enter Your Choice : 7